

WEEK 3

CYBERSECURITY

Advanced Network Security Practical Assessment

Student	Seyed Ismai Bilal S
Student ID	016
Role	Cybersecurity Intern
Target	Metasploitable 2 — 192.168.56.101
Network	VirtualBox Host-Only — 192.168.56.0/24
Tools	Kali Linux • Nmap • Wireshark
Submission	7 September 2026

Evidence-driven workflow: Discover → Enumerate → Capture → Analyze → Assess
Risk → Harden → Verify → Document

Executive Summary

This report documents an authorized internal-lab security assessment of Metasploitable 2. The assessment moved beyond command execution: each major stage produced evidence, the evidence was interpreted, high-risk conditions were prioritized, hardening was applied, and the resulting state was re-tested.

Active hosts discovered	3 hosts: 192.168.56.1, .100 and .101
Original TCP attack surface	23 open TCP ports
Normal traffic capture	38 packets; 0 dropped
Nmap investigation capture	Approximately 2,027 packets; 0 dropped
Hardening controls verified	FTP, Telnet and r-services
Verified post-hardening state	21/tcp, 23/tcp and 512–514/tcp closed

Assessment conclusion

The three selected hardening controls produced clear, measurable reduction in unnecessary network exposure. The final scan still identified legacy services that require further remediation; these are retained in the report as residual risk rather than being presented as fixed.

Scope

All active testing and configuration changes were restricted to the authorized VirtualBox lab network and target 192.168.56.101.

Contents & Task Traceability

Introduction & Objectives	Assessment purpose and success criteria
Lab Architecture & Tools	Environment, scope and methodology
Nmap Assessment	Task 7 — Network Discovery; Task 8 — Advanced Nmap
Wireshark Investigation	Task 9 — Capture; Task 10 — Nmap correlation; Task 11 — traffic investigation
Vulnerability & Risk Assessment	Task 12
Security Hardening	Task 13
Before/After & Final Assessment	Task 14
Recommendations & Conclusion	Final remediation and assessment outcome
Evidence Register	Screenshots, PCAPs and text outputs

Required submission items

The assignment requires a project folder, a 12–15 page PDF, an 8–12 slide presentation, a network diagram, screenshots, before/after evidence, Nmap/Wireshark artifacts, and a README. The GitHub repository must contain the project materials and must not contain passwords, private keys, API keys or credentials.

1. Introduction, Objectives & Lab Environment

The practical exercise simulates a small internal network where a security analyst must discover systems, understand exposed services, inspect traffic, identify weaknesses, reduce unnecessary exposure and provide evidence of improvement.

Objectives

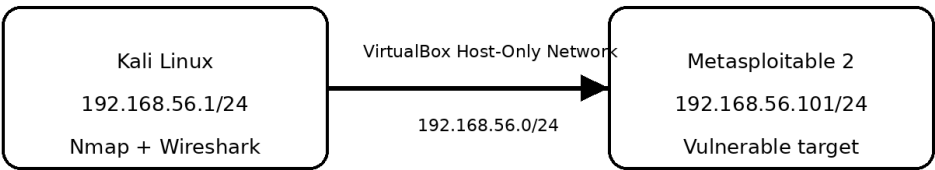
- Identify active systems inside the authorized assessment segment.
- Enumerate TCP and UDP services and determine service versions.
- Use Nmap NSE and service information to identify security weaknesses.
- Capture and inspect network traffic using Wireshark.
- Correlate Nmap behavior with packet-level evidence.
- Classify findings by risk with an evidence-based explanation.
- Implement at least three hardening improvements and verify them.
- Produce professional documentation suitable for technical review.

Lab environment

Assessment workstation	Kali Linux — 192.168.56.1/24
Target	Metasploitable 2 — 192.168.56.101/24
Network	VirtualBox Host-Only — 192.168.56.0/24
Capture interface	vboxnet0
Assessment boundary	Authorized lab network only

2. Network Architecture & Tools

Week 3 Cybersecurity Lab Architecture



Observed lab hosts during discovery:
192.168.56.1 | 192.168.56.100 | 192.168.56.101

Authorized assessment target: 192.168.56.101

Tools used

Nmap	Host discovery, port scanning, service/version detection, OS detection and NSE vulnerability checks
Wireshark	Packet capture, protocol analysis, endpoint statistics and TCP/DNS/HTTP investigation
Kali Linux	Assessment workstation and analysis environment
Metasploitable 2	Intentionally vulnerable target used only in the authorized lab

Methodology

The assessment followed a repeatable sequence: discover the network, enumerate services, inspect vulnerabilities, capture traffic, correlate observations, harden unnecessary services, re-scan, and document the change.

3. Nmap Assessment — Tasks 7 & 8

Host discovery identified three responding hosts. The authorized target was 192.168.56.101. A basic TCP scan identified 23 open ports, providing the baseline attack surface.

Key service groups

Remote access	22, 23	SSH; Telnet
Mail / DNS / Web	25, 53, 80	Postfix; BIND; Apache
Windows-style file services	139, 445	Samba
Legacy remote services	512, 513, 514	rexec, rlogin, rshd
Application / middleware	1099, 8009, 8180	Java RMI, AJP13, Tomcat
File / database	2049, 3306, 5432	NFS, MySQL, PostgreSQL
Remote desktop / GUI	5900, 6000	VNC, X11
Other services	2121, 6667, 1524	ProFTPD, UnrealIRCd, bind shell

Service/version detection also identified legacy software such as OpenSSH 4.7p1, Apache 2.2.8, MySQL 5.0.51a, PostgreSQL 8.3.x and ProFTPD 1.3.1.

3.1 Advanced Nmap Security Analysis

The advanced assessment used service/version detection, OS detection, TCP SYN scanning, limited UDP scanning and the Nmap vulnerability script set. The target presented a broad legacy attack surface.

High-value findings

- vsftpd 2.3.4 backdoor — CVE-2011-2523 reported vulnerable.
- TCP/1524 exposed a Metasploitable root bind shell.
- SSL POODLE, Logjam, weak Diffie-Hellman and OpenSSL CCS Injection findings were reported.
- UnrealIRCd backdoor/trojaned-version indication was reported.
- Java RMI registry class-loader / remote-code-execution risk was reported.
- Web enumeration exposed DVWA, Mutillidae, phpMyAdmin, webdav and phpinfo-related paths.
- HTTP TRACE and missing HttpOnly cookie attributes were observed.

UDP observations

The top-20 UDP scan reported UDP/53 as open DNS and several additional ports as open|filtered. Because open|filtered does not prove an active application, those results were treated cautiously in the analysis.

Analyst priority

Direct compromise conditions and unnecessary remote-access services were prioritized ahead of lower-impact configuration disclosures.

4. Wireshark Traffic Investigation — Tasks 9–11

Wireshark was run on vboxnet0. A normal traffic capture was retained separately from the Nmap investigation capture so protocol analysis and scan correlation remained easy to interpret.

Normal traffic	38 packets; 0 dropped; ARP, ICMP, TCP, HTTP, DNS and broadcast/NetBIOS-related traffic
Nmap investigation	Approximately 2,027 packets; 0 dropped; repeated TCP SYN probes, SYN/ACK responses and reset traffic

Protocol observations

- ICMP echo request/reply between Kali and Metasploitable.
- TCP handshake to HTTP/80, followed by HTTP GET and HTTP 200 OK.
- DNS query for metasploitable.local sent to 192.168.56.101:53; response indicated server failure.
- ARP traffic for local address resolution.
- Broadcast/NetBIOS-related traffic observed on the host-only segment.

4.1 Nmap + Wireshark Investigation

Task 10 linked the active Nmap scan to the packet-level behavior visible in Wireshark. This provided independent evidence that the scanner's results were generated by observable network exchanges.

TCP/80 open	SYN followed by SYN/ACK	Target accepted the connection attempt
Closed TCP ports	RST/ACK packets in the dedicated Nmap capture	Nmap used TCP response behavior to identify closed ports
HTTP service	GET followed by 200 OK	Application-layer communication confirmed
DNS service	UDP/53 query and response	DNS communication confirmed

Task 11 investigation note

Endpoint statistics showed Kali 192.168.56.1 and Metasploitable 192.168.56.101 as the main communicating endpoints. The normal traffic capture did not contain packets matching the TCP reset display filters; this was recorded as an absence rather than manufacturing evidence. The dedicated Nmap capture did contain reset traffic.

5. Vulnerability & Risk Assessment — Task 12

Risk ratings below use the assignment's Critical, High, Medium and Low categories. Each rating is tied to evidence and explains the potential impact.

vsftpd 2.3.4 backdoor	NSE scan; CVE-2011-2523	Critical	Known backdoor condition with potential remote compromise
Metasploitable bind shell	1524/tcp open	Critical	Direct remote shell exposure can lead to system compromise
Legacy r-services	512–514/tcp open	High	Insecure remote command/login services expand unauthorized-access risk
Telnet	23/tcp open	High	Legacy unencrypted remote administration
Legacy FTP	21/tcp open; vsftpd 2.3.4	High	Known vulnerable software exposed to the network
Legacy web/app stack	Apache 2.2.8, Tomcat, AJP13	High	Outdated components increase known-vulnerability exposure
SMB/NFS/databases	139/445, 2049, 3306, 5432	High	Remote data/file services increase attack and data-exposure surface
HTTP weaknesses	TRACE, cookie flags, exposed paths	Medium	Configuration weaknesses may support further attacks

6. Security Hardening — Task 13

Three practical improvements were implemented because they were unnecessary legacy services and produced clear before/after evidence.

FTP	21/tcp open; vsftpd 2.3.4	Disabled the FTP entry through xinetd configuration	21/tcp closed
Telnet	23/tcp open; telnetd	Commented the Telnet entry in /etc/inetd.conf	23/tcp closed
r-services	512–514/tcp open	Commented shell/login/exec entries in /etc/inetd.conf	512–514/tcp closed

Implementation note

The target uses an older inetd/xinetd service model. Modern service commands were unavailable in this lab image, so configuration-level hardening was used. The configuration was then reloaded and the ports were verified from Kali with Nmap.

Why these controls were selected

- They remove unnecessary legacy remote-access exposure.
- They are easy to verify objectively with a port scan.
- They demonstrate an actual reduction in attack surface.
- They align directly with the assignment's before → action → after requirement.

6.1 Before / After Evidence

Evidence was cropped to show the Metasploitable terminal only, removing the surrounding ChatGPT/browser area so the screenshots read as technical assessment evidence.

FTP baseline: 21/tcp was open.

```
tcp6 0 0 :::2121 :::* LISTEN
msfadmin@metasploitable:~$ su -c "netstat -tupln | grep ':21'"
Password:
su: Authentication failure
msfadmin@metasploitable:~$ su -c "netstat -tupln | grep ':21'"
Password:
su: Authentication failure
msfadmin@metasploitable:~$ grep -n "ftp" /etc/xinetd.conf /etc/xinetd.d/* 2>/dev/
null
/etc/xinetd.d/vsftpd:3:4 The vsftpd FTP server serves FTP connections. It uses
/etc/xinetd.d/vsftpd:5:4 vsftpd is designed to be secure.
/etc/xinetd.d/vsftpd:6:service ftp
/etc/xinetd.d/vsftpd:11: server = /usr/sbin/vsftpd
msfadmin@metasploitable:~$ ls -l /etc/xinetd.d/vsftpd
-rw-r--r-- 1 root root 576 2012-05-20 14:17 /etc/xinetd.d/vsftpd
msfadmin@metasploitable:~$ sudo -l
(sudo) password for msfadmin:
User msfadmin may run the following commands on this host:
(ALL) ALL
msfadmin@metasploitable:~$ sudo cp /etc/xinetd.d/vsftpd /etc/xinetd.d/vsftpd.bak
msfadmin@metasploitable:~$ sudo grep -n "disable" /etc/xinetd.d/vsftpd
16: disable
msfadmin@metasploitable:~$
```

Telnet configuration evidence in the legacy inetd configuration.

```
GNU nano 2.0.7 File: /etc/xinetd.conf Modified
#default settings
#(off) # methods=unix stream tcp nowait root /usr/sbin/tcpd /usr/sbin/
#(off) # stream tcp nowait telnetd /usr/sbin/tcpd /usr/sbin/in.tel
#(off) # ftp stream tcp nowait root /usr/sbin/tcpd /usr/sbin/
tftp dgram udp wait nobody /usr/sbin/tcpd /usr/sbin/in.tft
#shell stream tcp nowait root /usr/sbin/tcpd /usr/sbin/in.rsh
#login stream tcp nowait root /usr/sbin/tcpd /usr/sbin/in.rlogi
#exec stream tcp nowait root /usr/sbin/tcpd /usr/sbin/in.rex
loginsync stream tcp nowait root /bin/bash bash -l
```

FTP verification: 21/tcp was closed after hardening.

```
able = yes' /etc/xinetd.d/vsftpd
msfadmin@metasploitable:~$ sudo /etc/init.d/xinetd restart
sudo: /etc/init.d/xinetd: command not found
msfadmin@metasploitable:~$ sudo /etc/init.d/xinetd restart
* Stopping internet superserver xinetd [ OK ]
* Starting internet superserver xinetd [ OK ]
msfadmin@metasploitable:~$ grep -n "disable" /etc/xinetd.d/vsftpd
16: disable
msfadmin@metasploitable:~$ sudo sed -i 's/disable[:space:]]*=[:space:]]no/disable = yes' /etc/xinetd.d/vsftpd
msfadmin@metasploitable:~$ grep -n "disable" /etc/xinetd.d/vsftpd
16: disable = yes
msfadmin@metasploitable:~$ sudo /etc/init.d/xinetd restart
* Stopping internet superserver xinetd [ OK ]
* Starting internet superserver xinetd [ OK ]
msfadmin@metasploitable:~$ sudo grep -n "disable" /etc/xinetd.d/telnet
grep: /etc/xinetd.d/telnet: No such file or directory
msfadmin@metasploitable:~$ ls /etc/xinetd.d
charges logline discard echo time vsftpd vsftpd.bak
msfadmin@metasploitable:~$ grep -hnl "telnet" /etc/xinetd.conf /etc/xinetd.d/ 2>/dev/
null
msfadmin@metasploitable:~$ grep -n "telnet" /etc/xinetd.conf
2:telnet stream tcp nowait telnetd /usr/sbin/tcpd /usr/shi
u/in.telnetd
msfadmin@metasploitable:~$
```

6.2 r-services & Final Scan Evidence

r-services baseline: 512, 513 and 514 were open.

```
#-telnet stream tcp nowait telnetd /usr/sbin/tcpd /usr/s
n/w.telnetd
nsfadmin@kali:~$ sudo cp /etc/inetd.conf /etc/inetd.bak
nsfadmin@kali:~$ sudo sed -i 's/^#telnet/telnet/' /etc/inet
d.conf #added for nsfadm
nsfadmin@kali:~$ sudo systemctl restart
sudo: /etc/init.d/inetd: command not found
nsfadmin@kali:~$ sudo systemctl restart
sudo: /etc/init.d/inetd: command not found
nsfadmin@kali:~$ sudo service openedbsd-inetd restart
nsfadmin@kali:~$ ps aux | grep [l]net
root      7109  0.0  0.0 24284   864  ?        Ss   10:41:00 /usr/sbin/inet
d -pidfile=/var/run/inetd.pid -stayalive -inetd_compat
nsfadmin@kali:~$ sudo kill -HUP 7109
nsfadmin@kali:~$ grep -E "(^accept[ilist])" /etc/inetd.c
5:shell    stream  tcp    nowait root  /usr/sbin/tcpd  /usr/sbin/rs
shd
6:logins  d       stream  tcp    nowait root  /usr/sbin/tcpd  /usr/sbin/rs
shd
7:xcc     d       stream  tcp    nowait root  /usr/sbin/tcpd  /usr/sbin/rs
shd
cd
```

r-services verification: 512, 513 and 514 were closed.

```

1  #!/usr/bin/perl -w
2
3  use strict;
4
5  my $url = "http://www.google.com/";
6
7  my $response = get($url);
8
9  my $html = $response->content;
10
11  my $title = $html->title;
12
13  print "Title: $title\n";
14
15  my $body = $html->body;
16
17  print "Body: $body\n";
18
19  my $status = $response->status;
20
21  print "Status: $status\n";
22
23  my $headers = $response->headers;
24
25  print "Headers: $headers\n";
26
27  my $cookies = $response->cookies;
28
29  print "Cookies: $cookies\n";
30
31  my $encoding = $response->encoding;
32
33  print "Encoding: $encoding\n";
34
35  my $method = $response->method;
36
37  print "Method: $method\n";
38
39  my $url = $response->url;
40
41  print "URL: $url\n";
42
43  my $ip = $response->ip;
44
45  print "IP: $ip\n";
46
47  my $port = $response->port;
48
49  print "Port: $port\n";
50
51  my $host = $response->host;
52
53  print "Host: $host\n";
54
55  my $path = $response->path;
56
57  print "Path: $path\n";
58
59  my $query = $response->query;
60
61  print "Query: $query\n";
62
63  my $fragment = $response->fragment;
64
65  print "Fragment: $fragment\n";
66
67  my $scheme = $response->scheme;
68
69  print "Scheme: $scheme\n";
70
71  my $protocol = $response->protocol;
72
73  print "Protocol: $protocol\n";
74
75  my $version = $response->version;
76
77  print "Version: $version\n";
78
79  my $type = $response->type;
80
81  print "Type: $type\n";
82
83  my $mime = $response->mime;
84
85  print "MIME: $mime\n";
86
87  my $charset = $response->charset;
88
89  print "Charset: $charset\n";
90
91  my $language = $response->language;
92
93  print "Language: $language\n";
94
95  my $country = $response->country;
96
97  print "Country: $country\n";
98
99  my $region = $response->region;
100
101  print "Region: $region\n";
102
103  my $city = $response->city;
104
105  print "City: $city\n";
106
107  my $zip = $response->zip;
108
109  print "Zip: $zip\n";
110
111  my $lat = $response->lat;
112
113  print "Lat: $lat\n";
114
115  my $lon = $response->lon;
116
117  print "Lon: $lon\n";
118
119  my $time = $response->time;
120
121  print "Time: $time\n";
122
123  my $date = $response->date;
124
125  print "Date: $date\n";
126
127  my $time_zone = $response->time_zone;
128
129  print "Time Zone: $time_zone\n";
130
131  my $day_of_week = $response->day_of_week;
132
133  print "Day of Week: $day_of_week\n";
134
135  my $day_of_year = $response->day_of_year;
136
137  print "Day of Year: $day_of_year\n";
138
139  my $week_of_year = $response->week_of_year;
140
141  print "Week of Year: $week_of_year\n";
142
143  my $year = $response->year;
144
145  print "Year: $year\n";
146
147  my $month = $response->month;
148
149  print "Month: $month\n";
150
151  my $day = $response->day;
152
153  print "Day: $day\n";
154
155  my $hour = $response->hour;
156
157  print "Hour: $hour\n";
158
159  my $minute = $response->minute;
160
161  print "Minute: $minute\n";
162
163  my $second = $response->second;
164
165  print "Second: $second\n";
166
167  my $microsecond = $response->microsecond;
168
169  print "Microsecond: $microsecond\n";
170
171  my $nanosecond = $response->nanosecond;
172
173  print "Nanosecond: $nanosecond\n";
174
175  my $picosecond = $response->picosecond;
176
177  print "Picosecond: $picosecond\n";
178
179  my $femtosecond = $response->femtosecond;
180
181  print "Femtosecond: $femtosecond\n";
182
183  my $attosecond = $response->attosecond;
184
185  print "Attosecond: $attosecond\n";
186
187  my $zeptosecond = $response->zeptosecond;
188
189  print "Zeptosecond: $zeptosecond\n";
190
191  my $yoctosecond = $response->yoctosecond;
192
193  print "Yoctosecond: $yoctosecond\n";
194
195  my $xenosecond = $response->xenosecond;
196
197  print "Xenosecond: $xenosecond\n";
198
199  my $quattorsecond = $response->quattorsecond;
200
201  print "Quattorsecond: $quattorsecond\n";
202
203  my $quintosecond = $response->quintosecond;
204
205  print "Quintosecond: $quintosecond\n";
206
207  my $sextosecond = $response->sextosecond;
208
209  print "Sextosecond: $sextosecond\n";
210
211  my $septosecond = $response->septosecond;
212
213  print "Septosecond: $septosecond\n";
214
215  my $octosecond = $response->octosecond;
216
217  print "Octosecond: $octosecond\n";
218
219  my $nonasecond = $response->nonasecond;
220
219  print "Nonasecond: $nonasecond\n";
221
222  my $decasecond = $response->decasecond;
223
223  print "Decasecond: $decasecond\n";
224
225  my $centisecond = $response->centisecond;
226
226  print "Centisecond: $centisecond\n";
227
228  my $millisecond = $response->millisecond;
229
229  print "Millisecond: $millisecond\n";
230
231  my $microsecond = $response->microsecond;
232
232  print "Microsecond: $microsecond\n";
233
234  my $nanosecond = $response->nanosecond;
235
235  print "Nanosecond: $nanosecond\n";
236
237  my $picosecond = $response->picosecond;
238
238  print "Picosecond: $picosecond\n";
239
240  my $femtosecond = $response->femtosecond;
241
241  print "Femtosecond: $femtosecond\n";
242
243  my $attosecond = $response->attosecond;
244
244  print "Attosecond: $attosecond\n";
245
246  my $zeptosecond = $response->zeptosecond;
247
247  print "Zeptosecond: $zeptosecond\n";
248
249  my $yoctosecond = $response->yoctosecond;
250
250  print "Yoctosecond: $yoctosecond\n";
251
252  my $xenosecond = $response->xenosecond;
253
253  print "Xenosecond: $xenosecond\n";
254
255  my $quattorsecond = $response->quattorsecond;
256
256  print "Quattorsecond: $quattorsecond\n";
257
258  my $quintosecond = $response->quintosecond;
259
259  print "Quintosecond: $quintosecond\n";
260
261  my $sextosecond = $response->sextosecond;
262
262  print "Sextosecond: $sextosecond\n";
263
264  my $septosecond = $response->septosecond;
265
265  print "Septosecond: $septosecond\n";
266
267  my $octosecond = $response->octosecond;
268
268  print "Octosecond: $octosecond\n";
269
270  my $nonasecond = $response->nonasecond;
271
271  print "Nonasecond: $nonasecond\n";
272
273  my $decasecond = $response->decasecond;
274
274  print "Decasecond: $decasecond\n";
275
276  my $centisecond = $response->centisecond;
277
277  print "Centisecond: $centisecond\n";
278
279  my $millisecond = $response->millisecond;
280
280  print "Millisecond: $millisecond\n";
281
282  my $microsecond = $response->microsecond;
283
283  print "Microsecond: $microsecond\n";
284
285  my $nanosecond = $response->nanosecond;
286
286  print "Nanosecond: $nanosecond\n";
287
288  my $picosecond = $response->picosecond;
289
289  print "Picosecond: $picosecond\n";
290
291  my $femtosecond = $response->femtosecond;
292
292  print "Femtosecond: $femtosecond\n";
293
294  my $attosecond = $response->attosecond;
295
295  print "Attosecond: $attosecond\n";
296
297  my $zeptosecond = $response->zeptosecond;
298
298  print "Zeptosecond: $zeptosecond\n";
299
300  my $yoctosecond = $response->yoctosecond;
301
301  print "Yoctosecond: $yoctosecond\n";
302
303  my $xenosecond = $response->xenosecond;
304
304  print "Xenosecond: $xenosecond\n";
305
306  my $quattorsecond = $response->quattorsecond;
307
307  print "Quattorsecond: $quattorsecond\n";
308
309  my $quintosecond = $response->quintosecond;
310
310  print "Quintosecond: $quintosecond\n";
311
312  my $sextosecond = $response->sextosecond;
313
313  print "Sextosecond: $sextosecond\n";
314
315  my $septosecond = $response->septosecond;
316
316  print "Septosecond: $septosecond\n";
317
318  my $octosecond = $response->octosecond;
319
319  print "Octosecond: $octosecond\n";
320
321  my $nonasecond = $response->nonasecond;
322
322  print "Nonasecond: $nonasecond\n";
323
324  my $decasecond = $response->decasecond;
325
325  print "Decasecond: $decasecond\n";
326
327  my $centisecond = $response->centisecond;
328
328  print "Centisecond: $centisecond\n";
329
330  my $millisecond = $response->millisecond;
331
331  print "Millisecond: $millisecond\n";
332
333  my $microsecond = $response->microsecond;
334
334  print "Microsecond: $microsecond\n";
335
336  my $nanosecond = $response->nanosecond;
337
337  print "Nanosecond: $nanosecond\n";
338
339  my $picosecond = $response->picosecond;
340
340  print "Picosecond: $picosecond\n";
341
342  my $femtosecond = $response->femtosecond;
343
343  print "Femtosecond: $femtosecond\n";
344
345  my $attosecond = $response->attosecond;
346
346  print "Attosecond: $attosecond\n";
347
348  my $zeptosecond = $response->zeptosecond;
349
349  print "Zeptosecond: $zeptosecond\n";
350
351  my $yoctosecond = $response->yoctosecond;
352
352  print "Yoctosecond: $yoctosecond\n";
```

Final mini-assessment scan showing the remaining exposed services.

[illegible]

7. Final Mini Security Assessment — Task 14

The final assessment combines discovery, traffic analysis, security findings and hardening into one internal-network review.

What devices were identified?	192.168.56.1, 192.168.56.100 and 192.168.56.101; the target was .101.
What services were exposed?	The original target exposed 23 TCP services across remote access, web, databases, file sharing and legacy applications.
What protocols were observed?	ARP, ICMP, TCP, HTTP, DNS and broadcast/NetBIOS-related traffic.
Highest-risk findings?	vsftpd backdoor and the exposed bind shell; legacy remote-access services were also high risk.
What changed after hardening?	FTP, Telnet and r-services changed from open to closed.
Did the attack surface reduce?	Yes. Three unnecessary service groups were removed from network exposure; residual legacy services remain for future remediation.

Professional recommendations

- Remove remaining unnecessary legacy services, especially the bind shell and RMI exposure.
- Replace unsupported software with maintained versions and establish patch management.
- Restrict database, file-sharing and management services to trusted networks.
- Harden web/application configuration and remove unnecessary test applications.
- Repeat baseline Nmap/Wireshark reviews to detect unexpected exposure and configuration drift.

8. Submission Structure, Evidence Register & Conclusion

Yes — the required folder structure is part of the assignment submission. It is not merely an optional organizational suggestion. The assignment explicitly asks for one project folder with the following structure.

01-Nmap/Scans/	Nmap scan outputs
01-Nmap/Screenshots/	Nmap evidence screenshots
02-Wireshark/PCAP/	Wireshark .pcap/.pcapng files
02-Wireshark/Screenshots/	Wireshark evidence screenshots
03-Vulnerability-Assessment/	Vulnerability and final assessment outputs
04-Hardening/Before/	Pre-hardening evidence and configuration screenshots
04-Hardening/After/	Post-hardening evidence and verification
Network-Diagram/	Network topology diagram
Week-3-Report.pdf	12–15 page final report
Week-3-Presentation.pptx	8–12 slide presentation
README.md	Project overview, objective, lab, tools, tasks, findings, improvements, before/after and conclusion

Conclusion

The assessment completed the required evidence-driven workflow. The most important measurable outcome was the successful closure of FTP, Telnet and r-services exposure. The project artifacts are organized so the same evidence can be submitted through the report, presentation and GitHub repository without relying on screenshots alone.